



Course Details		
Class: II B. Tech	Semester: I	Academic Year: 2026-27
Course Title: ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS		
Course Code: 23A05302T		
Credits: 3 Regulation: NECR BTECH-23		
Program/Dept.: UG/ CSE-A, B, C, D, E, F AI&ML		
Faculty: Mr.A.Ramesh		

UNIT-I: Algorithm Analysis, AVL Trees, Heap Trees

1. What is an algorithm analysis.
Algorithm analysis is the process of determining the computational complexity of algorithms, i.e., the amount of resources (such as time and space) necessary to execute them.
2. What is time and space complexity of an algorithm?
Time Complexity: It measures the amount of computational time an algorithm requires as a function of the size of the input. It provides an upper bound on the running time of the algorithm, typically expressed using Big O notation (e.g., $O(n)$, $O(n^2)$). Time complexity helps to evaluate how the algorithm's execution time grows with the size of the input. Space Complexity: It measures the amount of memory space an algorithm needs as a function of the size of the input. It accounts for both the space used by the algorithm itself (e.g., variables, recursion stack) and the space needed for input and output. Space complexity is also typically expressed using Big O notation (e.g., $O(n)$, $O(n^2)$).
3. Describe the term "asymptotic analysis."
Asymptotic analysis is the study of an algorithm's efficiency as the input size approaches infinity. It focuses on identifying the dominant factors that dictate the algorithm's growth rate, while disregarding constant factors and less significant lower-order terms. This approach allows for a clearer understanding of how an algorithm scales with large inputs, providing a more meaningful comparison of different algorithms' performance in the long run.
4. Explain the significance of analyzing an algorithm's best-case, worst-case, and average-case time complexities.
Analyzing an algorithm's best-case, worst-case, and average-case time complexities is crucial

for understanding its overall performance.

- The best-case analysis shows the minimum time required, helping to identify the most efficient scenario.
- The worst-case analysis provides an upper limit, ensuring the algorithm's performance will not degrade beyond this point.
- Average-case analysis offers insight into the expected performance under typical conditions.
- Together, these analyses help in making informed decisions about the suitability and efficiency of algorithms for various applications.

5. What is an AVL tree and why is it important in computer science?

- An AVL tree is a self-balancing binary search tree where the difference in heights between the left and right subtrees of any node is at most one.
- This balance ensures $O(\log n)$ time complexity for insertion, deletion, and search operations, making AVL trees efficient for dynamic data structures that require frequent updates.

6. Describe the condition that must be met for an AVL tree to remain balanced.

- For an AVL tree to remain balanced, the height difference (or balance factor) between the left and right subtrees of any node must not exceed one.
- This condition ensures that the tree maintains $O(\log n)$ height, which is crucial for efficient operations.

7. What is an advantage of a balanced AVL tree and a disadvantage of an imbalanced AVL tree?

Advantage of a Balanced AVL Tree: A balanced AVL tree ensures that operations such as insertion, deletion, and search have a time complexity of $O(\log n)$, due to its height being logarithmic relative to the number of nodes, leading to efficient performance even with frequent updates.

Disadvantage of an Imbalanced AVL Tree: An imbalanced AVL tree, although rare, can suffer from degraded performance where operations approach $O(n)$ time complexity, as it fails to maintain the logarithmic height property, thus compromising the efficiency of the tree.

8. List out the operations of AVL Trees?

The following are operations of AVL trees:

1. Insertion
2. Deletion
3. Search
4. Traversal
 - Inorder Traversal
 - Preorder Traversal
 - Postorder Traversal
5. Find Minimum
6. Find Maximum
7. Get Height

8. Get Balance Factor
9. What is a heap tree and what are its main properties?
• A heap tree is a complete binary tree that satisfies the heap property. • In a max-heap, for any given node, the value of the node is greater than or equal to the values of its children, while in a min-heap, the value of the node is less than or equal to the values of its children. • The main properties are completeness (all levels are fully filled except possibly the last level) and the heap property (max-heap or min-heap condition).
10. Explain the difference between a max-heap and a min-heap.
• In a max-heap, the value of each node is greater than or equal to the values of its children, making the maximum value available at the root. • In a min-heap, the value of each node is less than or equal to the values of its children, making the minimum value available at the root.

UNIT-II: Graphs & Divide and Conquer

1. Why do we need graph data structure?

- Graph is another important non-linear data structure.
- In tree structure, there is a hierarchical relationship between parent and children, that is, one parent and many children.
- On the other hand, in graph, relationship is less restricted. Here, relationship is from many parents to many children.
- The below figure represents the two non-linear data structures.

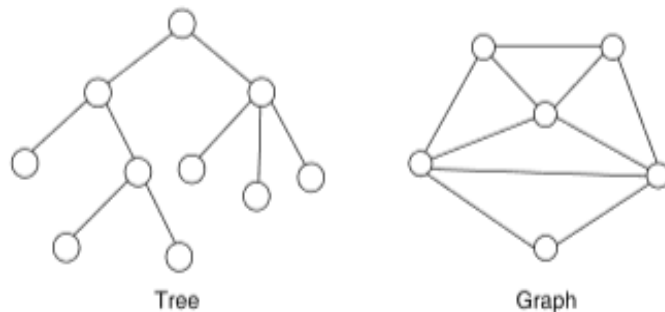


Figure Two non-linear data structures: tree and graph.

- Graph data structures are essential for representing and analyzing relationships between entities, such as in social networks or routing systems. They efficiently solve problems like shortest path finding and network connectivity.

2. What is the degree of a vertex in a graph?

- The degree of a vertex in a graph is the number of edges incident to it.
- In an undirected graph, it is the count of edges connected to the vertex.
- In a directed graph, there are two types of degrees: in-degree (the number of incoming edges) and out-degree (the number of outgoing edges).

3. What are the different representations of a graph, and what is their importance?

1. Set Representation:

- **Description:** Each edge is represented as a pair of vertices, and the set of edges is used to store all connections. For weighted graphs, pairs can include weights.
- **Importance:** Useful for algorithms that need to process all edges directly, such as those for minimum spanning trees or edge-centric computations.

2. Linked Representation:

- **Description:** Uses linked lists or similar structures to store adjacency information. Each vertex has a list of its adjacent vertices (or edges). In the case of a weighted graph, each

entry in the list also includes the weight of the edge. • Importance: Efficient in terms of space for sparse graphs and allows quick traversal of neighbors, making it ideal for dynamic graphs where edges are frequently updated. 3. Sequential (Matrix) Representation: • Description: Represents the graph using an adjacency matrix or edge matrix. For an adjacency matrix, a 2D array is used where each cell (i, j) indicates the presence (and possibly weight) of an edge between vertices i and j. • Importance: Provides fast access to edge information and is useful for dense graphs, but can be space-inefficient for sparse graphs due to its $O(V^2)$ space complexity.
4. What is a key difference in the traversal of a graph between BFS and DFS in terms of their approach? A key difference between BFS and DFS is their approach to exploring vertices: • BFS explores vertices in a level-order fashion, systematically visiting all neighbors of a vertex before moving to the next level. • DFS explores vertices as deeply as possible along one branch before backtracking and exploring other branches. This leads to different traversal orders and can affect the algorithms used for solving graph-related problems.
5. What is Breadth-First Search (BFS) and how does it differ from Depth-First Search (DFS)? • Breadth-First Search (BFS) is a graph traversal algorithm that explores all the vertices at the present depth level before moving on to the vertices at the next depth level. • BFS uses a queue to keep track of vertices. • Depth-First Search (DFS), on the other hand, explores as far as possible along each branch before backtracking, using a stack (or recursion) for its traversal. • The key difference is that BFS explores level by level, while DFS explores deeper into each branch.
6. What are connected components and biconnected components in graphs, and how do they differ? Connected Component: A connected component in a graph is a maximal subgraph in which any two vertices are connected by paths, ensuring that there is no additional vertex in the graph connected to this subgraph. It represents a subset of the graph where all vertices are reachable from each other. Biconnected Component: A biconnected component is a maximal subgraph where any two vertices are connected by at least two disjoint paths, meaning the component remains connected even if any single vertex is removed. It highlights regions in the graph that are resilient to vertex removal. The key difference is that connected components focus on reachability within a subset of vertices, while biconnected components ensure that the subset remains connected despite the removal of any single vertex.
7. What is the general idea behind the Divide and Conquer method? • The Divide and Conquer method involve breaking a problem into smaller, more manageable subproblems, solving each subproblem recursively, and then combining the solutions of the

subproblems to solve the original problem. • This approach is used to improve efficiency and simplify problem-solving.
8. What is the time complexity of Merge Sort?
The time complexity of Merge Sort is $O(n \log n)$ Where, n is the number of elements in the array. This is due to the divide-and-conquer approach of splitting the array and merging the sorted halves.
9. What is the primary advantage of Strassen's matrix multiplication algorithm over traditional methods?
The primary advantage of Strassen's algorithm is its reduced time complexity compared to traditional matrix multiplication methods. By minimizing the number of multiplication operations, Strassen's algorithm can be more efficient for large matrices.
10. How does the Divide and Conquer approach apply to Convex Hull algorithms?
In Convex Hull algorithms, the Divide and Conquer approach involves dividing the set of points into smaller subsets, recursively computing the Convex Hull for each subset, and then merging the results to form the Convex Hull of the entire set. This method is used to handle large sets of points efficiently.

UNIT-III: Greedy Method, Single Source Shortest Paths, and Dynamic Programming

1. What is the main difference between the Fractional Knapsack Problem and the 0/1 Knapsack Problem?

The main difference between the Fractional Knapsack Problem and the 0/1 Knapsack Problem is that in the Fractional Knapsack Problem, items can be divided into fractions and partially included in the knapsack. In contrast, the 0/1 Knapsack Problem requires items to be included in whole or not at all, with no partial inclusion allowed.

2. In the context of the Job Sequencing problem with deadlines, what criteria are used to select jobs in the Greedy approach?

In the Job Sequencing problem with deadlines, the Greedy approach selects jobs based on their highest profit, aiming to schedule them within their respective deadlines. The goal is to maximize total profit while ensuring each job fits within its deadline.

3. Explain the basic principle of the Greedy Method and provide one example where it is applied.

The basic principle of the Greedy Method is to make the locally optimal choice at each step with the hope of finding a global optimum, without reconsidering previous decisions. This approach involves selecting the best immediate option available at each stage. An example of the Greedy Method is Dijkstra's algorithm, which efficiently finds the shortest path from a source node to all other nodes in a graph by always choosing the shortest edge.

4. What is Kruskal's algorithm, and how does it relate to the Greedy Method?

Kruskal's algorithm is a Greedy algorithm used to find the Minimum Spanning Tree of a graph. It works by selecting the shortest edge that does not form a cycle, continuing until all vertices are connected.

5. Explain the key difference between a Binary Search Tree and an Optimal Binary Search Tree.

A Binary Search Tree (BST) organizes keys for fast lookup, while an Optimal Binary Search Tree (OBST) is structured to minimize the expected cost of searches, considering the probabilities of accessing each key.

6. What is the time complexity of Dijkstra's algorithm using a Min Heap?

The time complexity of Dijkstra's algorithm using a Min Heap is $O((V+E)\log V)$

Where,

V is the number of vertices

E is the number of edges.

This complexity accounts for the operations performed to update and retrieve vertices in the Min Heap.
7. What is the main principle behind Dijkstra's algorithm for finding the shortest path in a graph?
Dijkstra's algorithm finds the shortest path from a single source vertex to all other vertices in a graph by iteratively selecting the vertex with the minimum distance and updating the distances of its adjacent vertices.
8. What is the key principle behind Dynamic Programming?
The key principle behind Dynamic Programming is to decompose a complex problem into simpler subproblems, solve each subproblem only once, and store their solutions. This approach avoids redundant calculations by reusing previously computed results, leading to more efficient problem-solving.
9. How does Dynamic Programming differ from the Divide and Conquer approach?
Dynamic Programming solves each subproblem only once and stores the results to avoid redundant calculations. In contrast, Divide and Conquer breaks a problem into subproblems, solving each one independently and potentially solving the same subproblem multiple times without storing intermediate results.
10. In the context of All Pairs Shortest Paths, what does the Floyd-Warshall algorithm compute at each step?
The Floyd-Warshall algorithm computes the shortest paths between all pairs of vertices by iteratively updating the path lengths. At each step, it checks whether a shorter path exists through an intermediate vertex and updates the shortest path values accordingly. This process is repeated for each vertex as an intermediate point.

UNIT-IV: Dynamic programming & Backtracking

1. What is the 0/1 Knapsack problem in Dynamic Programming?
The 0/1 Knapsack problem is an optimization problem where you must select a subset of items, each with a weight and value, to maximize the total value without exceeding a given weight limit. Each item can either be included (1) or excluded (0).
2. How does Dynamic Programming solve the Travelling Salesperson Problem?
Dynamic Programming solves the Travelling Salesperson Problem (TSP) by evaluating all subsets of cities and computing the minimum cost to visit each subset. It stores these costs to avoid redundant calculations and builds up the final solution by combining the results of smaller subproblems.
3. What is the time complexity of solving the Travelling Salesperson Problem using Dynamic Programming?
• The time complexity of solving the Travelling Salesperson Problem (TSP) using Dynamic Programming is $O(n^2 \cdot 2^n)$, where n is the number of cities. • This complexity arises because the algorithm evaluates all subsets of cities, totaling 2^n subsets. • For each subset, it performs computations involving all cities, resulting in $O(n^2)$ operations per subset. This leads to the overall time complexity of $O(n^2 \cdot 2^n)$.
4. How does Dynamic Programming solve the String Editing problem?
Dynamic Programming solves the String Editing problem by constructing a table that records the minimum edit distance for all prefixes of the two strings, allowing for optimal solutions to be built from smaller subproblems.
5. Why is Dynamic Programming preferred over recursion in solving the 0/1 Knapsack problem?
Dynamic Programming is preferred because it avoids redundant calculations by storing the results of subproblems, leading to a more efficient solution compared to the exponential time complexity of the recursive approach.
6. What is the basic idea behind the backtracking algorithm?
The basic idea behind the backtracking algorithm is to build a solution incrementally, abandoning a solution as soon as it is determined that it cannot lead to a valid solution, and then backtracking to try other possibilities.
7. What is the 8-Queens problem?
The 8-Queens problem is a classic puzzle where the goal is to place 8 queens on a 8x8 chessboard such that no two queens can attack each other, meaning no two queens share the same row, column,

or diagonal.

8. How does backtracking help in solving the Sum of Subsets problem?

Backtracking helps solve the Sum of Subsets problem by incrementally exploring each subset and discarding any subset that exceeds the target sum. This approach reduces unnecessary calculations by focusing only on promising subsets that may meet the target, leading to more efficient problem-solving.

9. What is the difference between Dynamic Programming and Backtracking approaches in solving the 0/1 Knapsack problem?

The difference is that Dynamic Programming systematically computes and stores the solutions to subproblems, avoiding redundant work, while Backtracking explores possible solutions by trial and error, often revisiting the same subproblems without storing results.

10. How is backtracking applied to the Graph Coloring problem?

In the Graph Coloring problem, backtracking involves assigning colors to vertices one at a time. If a color assignment results in a conflict with adjacent vertices, the algorithm backtracks and tries a different color. This process continues until a valid coloring is found or all options are exhausted.

UNIT-V: Branch and Bound, NP Hard and NP Complete Problems, NP Hard Graph Problems, and NP Hard Scheduling Problems

1. What is the main idea behind the Branch and Bound method?
The main idea behind Branch and Bound is to systematically explore the solution space by dividing it into smaller subproblems (branching) and using bounds to eliminate subproblems that cannot produce a better solution than the current best.
2. How does Branch and Bound differ from the Backtracking approach?
Branch and Bound differs from Backtracking by using bounds to prune the search space more effectively, potentially eliminating entire branches without exploring them, whereas Backtracking relies on a depth-first search approach with limited pruning.
3. How is the Branch and Bound method applied to the Travelling Salesperson Problem (TSP)?
Branch and Bound is applied to TSP by systematically exploring possible routes, calculating bounds on the minimum possible cost, and pruning branches where the cost exceeds the current best solution.
4. What is the significance of Cook's Theorem?
Cook's Theorem is significant because it was the first to prove that the Boolean satisfiability problem (SAT) is NP-Complete, establishing the concept of NP-Completeness and the framework for classifying other problems as NP-Complete.
5. What is the difference between NP and NP-Complete problems?
NP problems are decision problems whose solutions can be verified in polynomial time, while NP-Complete problems are the hardest problems in NP, to which all other NP problems can be reduced in polynomial time.
6. What is the Clique Decision Problem (CDP) in graph theory?
The Clique Decision Problem involves determining whether a given graph contains a clique of a specified size, where a clique is a subset of vertices such that every two distinct vertices are connected by an edge.
7. Why is the Chromatic Number Decision Problem classified as NP-Hard?
The Chromatic Number Decision Problem is classified as NP-Hard because finding the minimum number of colors needed to color a graph is a computationally intensive task and is at least as hard as the hardest problems in NP.
8. Why is the Traveling Salesperson Decision Problem NP-Hard?

The Traveling Salesperson Decision Problem is NP-Hard because finding the shortest possible route that visits all cities and returns to the starting point is computationally challenging, and solving it is at least as hard as the hardest problems in NP.

9. What is the Job Shop Scheduling problem?

The Job Shop Scheduling problem involves scheduling a set of jobs, each requiring specific machines in a specific order, on multiple machines to optimize a certain objective, such as minimizing the total completion time or the makespan.

10. Why is the Job Shop Scheduling problem considered NP-Hard?

The Job Shop Scheduling problem is considered NP-Hard because finding the optimal schedule that minimizes the makespan or meets other criteria involves a large number of potential job-machine sequences, making it computationally difficult.

COURSE CO-ORDINATOR

COURSE-INSTRUCTOR